

Project Title

Unsupervised Learning with Dimensionality Reduction and Clustering

Project Notebook:

Created by: Koustab Ghosh & Sujoy Kumar Biswas

Designation:

1. Researcher: IDEAS-TIH, Indian Statistical Institute, Kolkata.
2. Head of Research & Innovation, IDEAS-TIH, Indian Statistical Institute, Kolkata.

Name of Student: NAYAN CHANDRA ROY

Name of University: VISVA BHARATI, SANTINIKETAN

Course Year: 2024-2028

Course Name: B.Sc (Hon's) in Statistics

Period of Internship: 18th May 2026 – 14th June 2026

Report submitted to: IDEAS – Institute of Data
Engineering, Analytics and Science Foundation, ISI
Kolkata

Project Note: Unsupervised Learning with
Dimensionality Reduction and Clustering

1. Abstract

This project explores the application of unsupervised learning techniques, specifically K-Means clustering and Principal Component Analysis (PCA), to a dataset of handwritten digits. The study demonstrates how high-dimensional image data (64 features) can be reduced to a lower-dimensional space (2 features) and grouped into clusters that represent different numerical digits without initial supervision. Performance is evaluated by mapping unsupervised clusters to their most frequent true labels and calculating classification metrics.

2. Introduction

The analysis utilizes the handwritten digits dataset from the sklearn library, which contains 1,797 samples. Each sample is an 8x8 pixel image, resulting in 64 dimensions per digit. The primary challenge addressed is identifying patterns within this high-dimensional space and determining if unsupervised methods can effectively distinguish between the digits 0 through 9.

3. Project Objective

- i. Identify natural groupings in the handwritten digits data using K-Means clustering.
- ii. Reduce dimensionality of the dataset from 64 features to 2 features using PCA to simplify the data structure.
- iii. Implement a label mapping strategy to align unsupervised cluster labels with actual digit identities.
- iv. Evaluate the effectiveness of the clustering through accuracy scores, confusion matrices, and detailed classification reports.

4. Methodology

- i. Data Acquisition: Load the digits dataset, which consists of 1,797 samples with a shape of (1797, 64)
- ii. Initial Clustering: Apply K-Means with 10 clusters (representing digits 0-9) directly to the 64-dimensional data.
- iii. Dimensionality Reduction: Utilize PCA to transform the 64-dimensional data into a 2D representation (reduced_data).
- iv. Secondary Clustering: Perform K-Means again, this time on the 2D reduced_data.
- v. Cluster Label Mapping: Use `scipy.stats.mode` to find the most common true label within each cluster, creating a mapping (labels_map) to convert cluster IDs to actual digit predictions.

vi. Performance Evaluation: Compare the predicted mapped labels (pred_labels) against true labels using accuracy and confusion matrices.

5. Data Analysis and Results

i. Data Structure: The original features were successfully reshaped from 64-element vectors into 8x8 images for inspection of cluster centers.

ii. PCA Impact: PCA reduced the data shape to (1797, 2), facilitating a simpler clustering environment.

iii. Clustering Outcomes: The model generated 10 cluster centers, with each center representing the "average" digit for that group.

iv. Evaluation Metrics: The performance was measured using accuracy_score and a confusion_matrix to identify which specific digits were most frequently confused by the unsupervised model. A full classification_report provided precision, recall, and F1-scores for each digit.

6. Conclusion

The project confirms that unsupervised techniques like K-Means and PCA can effectively organize handwritten digit data. By reducing the data to two dimensions and mapping clusters to their statistical models, the model can achieve measurable accuracy in digit recognition. This approach highlights the power of unsupervised learning in finding latent structures in complex datasets.

7. Appendices

- i. Key Libraries: The project requires NumPy, SciPy, Stats, and Sklearn (specifically datasets, cluster, decomposition, and metrics).
- ii. Note on Environment: The source documentation indicates that the execution environment initially lacked these modules, resulting in Module Not Found Error during the initial setup.
- iii. Cluster Center Reshaping: A critical step for visualization involved reshaping the (10, 64) cluster center array into a (10, 8, 8) 3D array.

Question 1: Load Libraries and Dataset (2 Marks)

Import the `load_digits` function from `sklearn.datasets`. Load the handwritten digits dataset into a variable named `digits`. Print the shape of the `digits.data` attribute.

```
# Question 1

from sklearn.datasets import load_digits

# Load the dataset
digits = load_digits()

# Print the shape of the data attribute
print(digits.data.shape)

(1797, 64)
```

Question 2: Perform K-Means Clustering (5 Marks)

Import `KMeans` from `sklearn.cluster`. Create an instance of the `KMeans` model with `n_clusters=10` and `random_state=0`. Fit this model to the `digits.data` and get the cluster predictions. Print the shape of the `cluster_centers_` attribute of your fitted model.

Expected Output:

```
(10, 64)

# Write your answer here
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans

# Load the dataset
digits = load_digits()

# Print the shape of digits.data
print("Shape of digits.data:", digits.data.shape)

# Create and fit K-Means model
kmeans = KMeans(
    n_clusters=10,
    random_state=0,
    n_init=10
)

clusters = kmeans.fit_predict(digits.data)

# Print the shape of cluster centers
print("Shape of cluster_centers_:", kmeans.cluster_centers_.shape)
```

```
Shape of digits.data: (1797, 64)
Shape of cluster_centers_: (10, 64)
```

Question 3: Inspect Cluster Centers (2 Marks)

The cluster centers represent the 'average' digit for each cluster. Reshape the `kmeans.cluster_centers_` into a 3D array of shape `(10, 8, 8)` and store it in a variable called `centers`. Print the shape of the first center (i.e., `centers[0]`).

Expected Output:

```
(8, 8)
# Write your answer here
centers = kmeans.cluster_centers_.reshape(10, 8, 8)
print(centers[0].shape)
(8, 8)
```

Question 4: Dimensionality Reduction with PCA (4 Marks)

Import PCA from `sklearn.decomposition`. Create a PCA model to reduce the 64-dimensional digit data to 2 dimensions. Fit and transform the `digits.data` with this model, storing the result in a variable named `reduced_data`. Print the shape of `reduced_data`.

Expected Output:

```
(1797, 2)
# Write your answer here
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
reduced_data = pca.fit_transform(digits.data)
print(reduced_data.shape)
(1797, 2)
```

Question 5: Clustering on Reduced Data (3 Marks)

Perform K-Means clustering again, but this time on the `reduced_data` from the previous step. Use `n_clusters=10` and `random_state=0`. Store the predictions in a variable `clusters`. Print the first 10 cluster labels.

Expected Output: A numpy array showing the first 10 predicted cluster labels.

```
# Write your answer here
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=10, random_state=0)

clusters = kmeans.fit_predict(reduced_data)

print(clusters[:10])

[9 4 5 6 2 6 2 8 3 3]
```

Question 6: Label Mapping (5 Marks)

Since K-Means does not know the actual digit labels, we need to map our cluster labels (0-9) to the most common true digit label within each cluster. Import `scipy.stats.mode`. For each cluster, find the mode of the true `digits.target` labels. Store this mapping in a dictionary called `labels_map`.

Hint: Loop from cluster 0 to 9. In each loop, find the mode of `digits.target[clusters == i]` where `i` is the cluster number.

Expected Output: A dictionary where keys are cluster labels and values are the corresponding most frequent true digit labels.

```
# Write your answer here
from scipy.stats import mode

labels_map = {}

for i in range(10):
    labels_map[i] = mode(
        digits.target[clusters == i],
        keepdims=True
    ).mode[0]

print(labels_map)

{0: np.int64(2), 1: np.int64(0), 2: np.int64(4), 3: np.int64(8), 4:
np.int64(1), 5: np.int64(8), 6: np.int64(9), 7: np.int64(6), 8:
np.int64(7), 9: np.int64(0)}
```

Question 7: Creating Final Predictions (3 Marks)

Using the `labels_map` dictionary from the previous step, create an array of predicted labels `pred_labels` by converting your K-Means `clusters` array to the mapped digit labels. Print the first 10 predicted labels.

Expected Output: An array showing the first 10 final predicted digit labels.


```
# Write your answer here
import numpy as np

pred_labels = np.array(
    [labels_map[label] for label in clusters]
)

print(pred_labels[:10])

[0 1 8 9 4 9 4 7 8 8]
```

Question 8: Evaluate Clustering Accuracy (2 Marks)

Import `accuracy_score` from `sklearn.metrics`. Calculate and print the accuracy of your unsupervised clustering by comparing the `pred_labels` with the true `digits.target` labels.

Expected Output: A single floating-point number representing the accuracy score.

```
# Write your answer here
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(
    digits.target,
    pred_labels
)

print(accuracy)

0.5664997217584864
```

Question 9: Confusion Matrix (2 Marks)

Import `confusion_matrix` from `sklearn.metrics`. Compute and print the confusion matrix to see which digits were most often confused with each other.

Expected Output: A 10x10 numpy array representing the confusion matrix.

```
# Write your answer here
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
    digits.target,
    pred_labels
)

print(cm)

[[167  0  0  0  1  0  7  0  3  0]
 [ 2 97  0  0  4  0  0 11 68  0]
 [ 0  3 114  0  0  0  0 10 47  3]]
```

```
[ 2  0 63  0  0  0  0 13 15 90]
[ 0 11  0  0 138  0 27  3  2  0]
[14  6  4  0  2  0  4 23 104 25]
[22  0  0  0 21  0 136  0  2  0]
[ 0 35  0  0  1  0  0 115 28  0]
[ 0 10  3  0  1  0  0  24 136  0]
[ 4  1  8  0  0  0  1  9 42 115]]
```

Question 10: Model Evaluation with Classification Report (2 Marks)

Import `classification_report` from `sklearn.metrics`. Print the full classification report for your predicted labels against the true labels to get a detailed breakdown of precision, recall, and f1-score for each digit.

Expected Output: A text-based report showing precision, recall, and f1-score for each digit class (0 through 9).

```
# Write your answer here
from sklearn.metrics import classification_report

print(classification_report(
    digits.target,
    pred_labels
))
```

	precision	recall	f1-score	support
0	0.79	0.94	0.86	178
1	0.60	0.53	0.56	182
2	0.59	0.64	0.62	177
3	0.00	0.00	0.00	183
4	0.82	0.76	0.79	181
5	0.00	0.00	0.00	182
6	0.78	0.75	0.76	181
7	0.55	0.64	0.59	179
8	0.30	0.78	0.44	174
9	0.49	0.64	0.56	180
accuracy			0.57	1797
macro avg	0.49	0.57	0.52	1797
weighted avg	0.49	0.57	0.52	1797

```
C:\Users\nayan\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-
packages\Python313\site-packages\sklearn\metrics\
_classification.py:1879: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
```

```
result.shape[0])
C:\Users\nayan\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-
packages\Python313\site-packages\sklearn\metrics\
_classification.py:1879: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
C:\Users\nayan\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-
packages\Python313\site-packages\sklearn\metrics\
_classification.py:1879: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 in labels with no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
result.shape[0])
```